

AI 应用能力文档

出海达人营销投前决策智能体

纪辉（Hugh） | 天津大学 计算机科学与技术 2023 级

把一句话投放 brief，变成一份可执行、可追责、带预算分配和风险证据链的达人投放清单，并用可复现的实验说清这份清单比「凭粉丝量选人」少浪费多少钱——以及这个结论有多稳。

这份文档里的每一个数字都有出处：绝大部分能在仓库里用一条命令复跑（固定种子、无网络依赖、两次运行产物逐字节相同）；少数标明「服务实测」的来自 HTTP 服务，在仓库里 `cd api && python3 main.py` 起服务后可自己 `curl` 复核。

在线 Demo（浏览器直接打开，无需安装）：<https://2990709874-ui.github.io/koxpilot/>

代码仓库（本文档 PDF、全部源码、实验产物与六篇技术分册均在其中）：<https://github.com/2990709874-ui/koxpilot>

本地起前端：`cd web && pnpm install && pnpm run prepare-data && pnpm run dev`

HTTP 服务：`pip install -r api/requirements.txt` 后 `cd api && python3 main.py` 本地起（`api/run.sh`

是部署平台生成的脚本，本地不用它），无需鉴权即可 `curl`；部署包见 [deploy/hf-space/](#)

提交入口只有一个 GitHub 仓库：<https://github.com/2990709874-ui/koxpilot> | [在线 Demo](#)（无需安装，浏览器直接打开）：<https://2990709874-ui.github.io/koxpilot/>

HTTP 服务、本文档 PDF、全部源码与实验产物、六篇技术分册（docs/01~docs/06，约 3.4 万字）都在同一个仓库里，README 顶部有按阅读时长分流的导航。

0. 摘要

0.1 这是什么

KOXPilot 是一个[出海达人营销的投前决策智能体](#)：输入一句自然语言投放需求（例：「3C 小家电新品要投北美，8 万美元，看转化」），输出一份可执行、可追责的达人投放方案——选谁、买几条内容、每人给多少钱、为什么选他、风险在哪一条判据上。

它解决的不是投后优化，而是[投前选错人](#)造成的浪费：达人自称做 3C、实际内容在做美妆；粉丝 200 万但互动率 0.3%；受众 80% 在印尼而投放目标是沙特。这些问题在投放结束后看数据能发现，但钱已经花完了。

0.2 能力构成

环节	能力	实现方式
A1 需求解析	自然语言 → 结构化投放规格（品类 / 市场 / 平台 / 人群 / 预算 / 竞品回避 / 考核口径）	LLM 优先，凭据缺失或调用失败自动回落确定性规则解析
A2 候选召回	5,000 人达人库 → 命中定向的候选池	规则
A3 质量门禁	四层判定 G0/G1/G2/G3 → 可投放 / 需人工复核 / 不予投放 + 逐条证据链	规则 + 分组分位阈值
A4 适配打分	达人与本单需求的语义匹配度	正式链路为规则版；LLM 版构建期真调并留存对照（§ 6.2）
A5 预算分配	5 条硬约束下的金额与内容条数分配	规则 + 边际性价比贪心与约束修正
A6 事后核算	成本审计 + 反事实价值审计（与两条对照臂比）	规则，判定一律按标注结果结算

0.3 关键结果

维度	结果
水号识别（严口径）	P 0.7714 / R 0.6750 / F1 0.7200 / AUC 0.8809 ；12 种子稳定性 F1 0.7275 ± 0.0090 、AUC 0.8878 ± 0.0054 （CV ≤ 0.023）
三档判定	准确率 0.6590 / macro F1 0.6731；pass 精确率 0.9466 、review 召回 0.9613 （判据取向：宁可多送人工复核，不让水号直接过审）
业务价值（对照「按粉丝量降序选人」）	单次实测少浪费 32.5%、有效曝光 +98.64%； 12 种子稳健区间 -21.5% ± 13.7% （1/12 跑输基线），本文对外一律用后者

维度	结果
风险收敛	浪费率跨观测标准差 33.6% → 4.7%；浪费率 > 50% 的观测 9/36 → 0/36
模型成本	混合架构把 LLM 调用量从全 LLM 的 888 次降到 225 次 (1/3.95)；构建期真实 token 账 172 次调用 / 110 万 token（取自 API 返回的 usage）
工程	768 条 pytest 全绿；固定种子下 make eval 两次运行产物逐字节相同；Python 与 TypeScript 双实现判定级 5,000/5,000 差异 0

0.4 边界与限制（写在第一页，不放附录）

1. 数据是合成的，不是真实达人数据。所有绝对数值只在这套数据生成假设内成立，这是本作品最大的边界。
2. 阈值敏感性判定为「不稳健」：阈值 $\pm 20\%$ 时最大 F1 偏移 0.0582，产物里如实标 stable=False，没有调宽判据让它变 True。
3. 自己加的对照臂在主指标上胜出：只做结构分散、不做门禁的第三臂，每美元有效曝光在三条 brief 上分别高 47.4% / 47.5% / 44.0%。代价是 18.25% 预算打水漂、41 个水号、4 个品牌安全高风险号、228 份合约。门禁买的是确定性与品牌安全，不是每美元曝光效率的上限（§ 5.4）。
4. BRIEF-002 上门禁没起到应有作用：3 个水号进入最终名单，浪费率 12.21% 高于对照臂的 8.21%，是真实待改进项，不是口径问题。
5. 线上服务的 AI 当前走规则解析：演示实例未配置模型凭据，/api/health 与 /api/plan 响应逐字写明降级原因，parse_path=rule。模型路径本身已跑通并留有 token 账；配置凭据重启即走 LLM（§ 3.4）。
6. 服务是演示实例，不是生产部署：单实例、无副本、无灰度、只读不写，冷启动首个请求需等数据集懒加载（常温 /api/health 约 120ms）。

1. 场景与问题定义

1.1 为什么选这条链路

出海营销是一条钱密集、决策却高度依赖经验的链路，而达人 / 网红营销是其中投放占比最高的形态之一。本项目没有做成一个通用的「AI 助手」，而是聚焦这条链路上钱最容易白烧、且判断可以被结构化的环节：投前选人。

1.2 为什么指标选「少烧」而不是「多赚」

投放代理是低毛利生意。在毛利率只有个位数的业务里，「帮你多赚 10%」很难证明，「帮你少烧 10%」直接落在利润上。这个判断决定了后面所有指标的选择：主口径是浪费金额与有效曝光，而不是效果提升。

2. 产品能力

2.1 决策链路

一句话投放需求（自然语言）

- ↓ A1 需求解析 — 自然语言 → 结构化投放规格；模型不可用时走确定性规则解析
- ↓ A2 候选召回 — 按品类 / 地域 / 平台 / 粉丝层召回
- ↓ A3 质量门禁 — 四层判定 G0/G1/G2/G3 → 可投放 / 需人工复核 / 不予投放 + 证据链
- ↓ A4 适配打分 — 达人与本单需求的语义匹配度
- ↓ A5 预算分配 — 5 条硬约束下的金额与条数分配
- ↓ A6 事后核算 — 成本审计 + 反事实价值审计
- ↓

可执行方案：选谁、买几条、花多少、为什么、风险在哪

- └ 预算无法用尽时：给出原因结论 + 可执行的放宽建议（§ 2.4）

2.2 四层质量门禁

层	管什么	典型判据
G0 资料完整性	关键字段缺失或过期	数据不足时不给结论，而不是硬给一个
G1 数据真实性	水号识别	互动率与同组分位背离、粉丝增长毛刺、评论质量、播放 / 粉丝比异常
G2 人群匹配度	声称与实际是否一致	声称品类 vs 实际内容、受众地域 / 语言与投放目标是否错配
G3 品牌安全	品牌风险敞口	敏感品类、争议历史、儿童向标准等

关键设计：阈值按分组标定，不是全局写死。instagram 头部号与 youtube

腰部号的「正常互动率」不在一个量级，全局阈值会同时误杀一批、放过一批。所以阈值按平台 × 粉丝层分组、用分位数标定，样本不足的组显式降级并在产物里上报。

2.3 输出的是可追责的判定，不是一个分数

每个被拒或被标记复核的达人都带证据链：命中哪条规则、该分组的分位阈值是多少、他的实测值是多少。这是产品化的关键——投放执行的人不会因为「模型给了 0.31 分」砍掉一个达人，但会因为「他的互动率在同组里排后 3%，且评论有 60% 是模板句」而砍。

2.4 预算无法用尽时，给建议而不是交白卷

严门禁必然带来一个后果：定向越窄、过审的人越少，预算越可能花不完。筛选器到这里就结束了；决策产品需要继续往下走一步。

因此当预算利用率低于 60% 时，产品先给出结论块说明未用尽的原因，再给最多三条可执行的放宽建议。每条建议只改一处、其余定向不动，并且是在放宽后的规格上真跑一遍 A2→A3→A5 算出来的，不是估算。

以一条自由文本需求（巴西 + 墨西哥 Instagram 彩妆，18-24 女性，\$80,000，回避 Focallure）为例，服务实测召回 182 → 过门禁 7 → 进方案 4 人，仅花掉 \$1,249.75。结论是：通过门禁且报价与曝光齐备的候选只有 7 人，剩余额度已贴住结构配额（单一国家金额占比 ≤ 60%，当前 59.0%）。三条建议按增量可花金额降序：

放宽哪一处	增量可花	放宽后利用率	新增入选
加投相邻市场（BR、MX → 扩到 AR、CO、US）	+\$43,291	55.68%	+24 人
允许「需人工复核」档带折扣进方案	+\$41,488	53.42%	+62 人
去掉 18-24 年龄段限定	+\$4,408	7.07%	+7 人

每条建议都带质量说明（新增达人的门禁判定、放宽的是定向还是采购动作、风险敞口多少）——**放宽不等于放水**，新进方案的人同样逐条过四层门禁。三条建议全部调用包内既有函数，服务层不新增任何判定或排序逻辑。

需要一并说清的前提：这条需求花不出去钱，是**门禁只放过 7 人**造成的，不是候选被截断。命中定向的 182 人全部进入门禁与预算计算（响应里 scope.computed_on = 182、truncated_for_compute = false）。三条预置需求的预算利用率都 ≥ 99.8%，所以这条自由文本需求的 1.6% 是「严门禁 + 窄定向」叠出来的极端样本，不是产品常态。

2.5 界面

界面是[单页结构](#)，按决策阅读顺序排列，不做功能分区的页签：

顺序	区块	回答什么
1	投放需求	这次要投什么、投哪、多少钱、怎么考核（可直接改写并重算）
2	决策结论	建议签约多少人、预算投入多少、判定不予投放多少、相较「按粉丝量投放」减少多少无效支出
3	候选筛查与拦截过程	5,000 人是怎么收敛到这份名单的，每一环节挡掉了谁、依据是哪条判据
4	投放名单与预算分配	每个人分到多少钱、买几条内容、适配度与风险标记
5	与两种常见投放方式的对照	与「按粉丝量投放」「结构分散但不筛查」在每美元有效曝光、浪费金额、风险敞口上的差异
6	实现说明（折叠）	当前算力通道、数据口径、复现入口

评测指标、双实现一致性、阈值标定口径等工程内容**不进入界面**，统一写在仓库 README 与 docs/。界面只回答「这次该签谁、花多少钱、为什么不签那些人」。

3. 系统实现与运行形态

3.1 一套业务内核，两条执行通道

Demo 不播放预录结果。页面首屏先探活 Python 服务，随后按可用性选择执行通道：

[首屏] 浏览器打开页面

└ 探活 GET /api/health (800ms 内需返回)

[服务可用] 通道 A：Python FastAPI 服务

- api/ 直接 import src/koxpilot, A1→A6 全链路真算
- 门禁 gates.engine / 预算 budget.planner / 审计 eval.audit / 证据句 gates.humanize
- 响应携带 parity_payload.verdicts (本次召回集每条 kox_id 的判定)

[服务不可用 / 超时 / 返回 ok:false] 通道 B：浏览器内 TypeScript 引擎

- 召回 → 门禁 → 预算 全流程在浏览器内完成，功能不缺项
- GitHub Pages 上的默认形态即为此通道

[服务可用时，两条通道同时运行]

- 浏览器引擎对同一批 kox_id 独立重算，与服务判定逐条比对
- 界面只显示一句结论（核对了多少人、结论是否一致），逐项对照写入 pnpm run verify 的产物

三点需要单独说明：

1. **服务不是壳。** api/koxpilot_service/ 只做参数校验与形状转换，门禁、预算、审计、证据句全部 import src/koxpilot——一套逻辑，不是为接口重写一遍。/api/kox/{id}/explain 的每一句理由与命令行 koxpilot explain 逐字一致，由 api/check_cli_parity.py 逐字符断言。
2. **两条通道各自不可替代。** 只留通道 A，就没有任何独立手段能证伪服务的输出；只留通道 B，就没有「服务端真跑」这件事。代价是两套实现不允许漂移，这也是每次构建都要跑一致性校验的原因。
3. **界面如实显示当前通道**（顶栏状态点三态：Python 服务 / 浏览器引擎 / 探活中）。降级时功能不允许缺失，这一条写在服务契约里。

3.2 双实现一致性校验

层级	口径	结果
运行期	真服务 vs 浏览器 TypeScript 引擎，4 条需求合计判定条数	1,431 条，差异 0
运行期	explain 证据句 vs 命令行输出，逐字比对	5 个 id，差异 0
构建期	全库判定级	5,000 / 5,000，差异 0
构建期	证据链级 / 预算级 / 审计级	255 条理由、9 个臂（3 需求 × 3 臂）、3 个 campaign，diff 全为 0

3.3 服务契约与计算范围

服务契约冻结在 api/CONTRACT.md，当前版本 v1.1 (engine_version 1.1.0)。/api/plan 响应含一个只读 scope 块，用于证明服务与离线命令行算的是同一道题：

字段	含义	BRIEF-001 实测
recall_total	命中定向的候选总数	746
computed_on	真正进入门禁与预算计算的人数（必须等于 recall_total）	746
detail_rows	返回给前端的候选明细条数	60
truncated_for_compute	计算是否被截断（恒为 false）	false

这四个字段是一条真实 bug 的直接产物：早期版本把展示参数 top_n 当成了召回上限，导致服务与离线对同一条需求算出的预算利用率差 14 倍（§ 9.3）。

3.4 大模型接入现状（实测）

链路里唯一可能在运行时调模型的是 A1 需求解析。它的契约是：探到可用凭据就真调，凭据缺失 / 调用失败 / 超时一律回落确定性规则解析，并把本次走的路径与降级原因写进响应，而不是吞掉。

当前公开演示实例的实测结果（可自行 curl 复核）：

```
GET /api/health
  "status": "ready"
  "engine": "python", "engine_version": "1.1.0", "kox_count": 5000
  "llm_runtime": { "available": false, "provider": null,
                  "reason": "未配置模型凭据, A1 使用确定性规则解析" }

POST /api/plan
  "brief": { "parse_path": "rule",
            "notes": ["未配置模型凭据, A1 使用确定性规则解析"] }
  "scope": { "recall_total": 182, "computed_on": 182, "truncated_for_compute": false }
```

准确的表述是：演示实例没有配置模型凭据，A1 因此走规则解析，降级原因由服务逐字写进响应。模型路径本身不是空壳——构建期在配置了凭据的环境里真实调用过 ARK（doubao-seed）与 Azure OpenAI（GPT-5.5）两家模型，token 账留在 output/llm_bench.json（172 次调用、110 万 token，数值取自 API 返回的 usage，非估算）。要让线上也走 LLM 路径，只需给服务环境配置 ARK_API_KEY 或 Azure 相关变量后重启，无需改代码。

同一条需求交给 LLM 解析与交给规则解析的差异也在产物里：30 个字段中 27 个一致，差异全部落在竞品回避一项——LLM 会按品类常识补出原文没写的竞品，规则版补不出来。

3.5 部署形态

形态	说明
本地 Demo	cd web && pnpm install && pnpm run prepare-data && pnpm run dev，不依赖任何外部服务
公网 Demo	仓库内已配好 GitHub Actions（.github/workflows/pages.yml），仓库 Settings → Pages 选 GitHub Actions 后自动发布；Pages 上走浏览器引擎通道，功能不缺项

形态	说明
本地服务	pip install -r api/requirements.txt 后 cd api && python3 main.py, 无鉴权, 可直接 curl
公网服务 (可选)	deploy/hf-space/ 是 Hugging Face Spaces 一键部署包 (已本地验证) ; 部署后改 web/public/api-config.json 的 apiBase 即可让前端接上, 无需改代码重建

服务是演示实例：单实例、无副本、无灰度、只读不写，冷启动首个请求需等数据集懒加载。它证明的是「这条链路能被 HTTP 真调、且与离线算同一道题」，不是「这套东西扛得住生产流量」。

4. 决策质量评测

评测口径：5,000 个合成达人，标注三档（可投放 3,058 / 需人工复核 1,034 / 不予投放 908）。

防自证设计：标注结果只允许评测代码读取，门禁、预算、LLM 正式链路在物理上读不到。仓库里有 AST 静态扫描与泄漏哨兵测试守着这条线——因为这个项目上出现过两次自证缺陷（§ 9.1）。

指标	结果
水号识别 (严口径)	P 0.7714 / R 0.6750 / F1 0.7200 / AUC 0.8809
三分类	准确率 0.6590 / macro F1 0.6731
可投放档	P 0.9466 / R 0.5386
需人工复核档	P 0.4052 / R 0.9613
不予投放档	P 0.8104 / R 0.7203

可投放档精确率 0.9466、复核档召回 0.9613 是刻意的取向：这套门禁的产品定位是「宁可多送一批人去人工复核，也不要让水号直接过审花钱」。因此接受复核档精确率只有 0.4052（多送人工），换取过审名单干净、疑似水号不漏。这不是指标没调好，是判据取向。

分欺诈类型的能力边界：买粉严召回 0.797 / AUC 0.919、互动农场 0.670 / 0.880、机器评论 0.640 / 0.893，而**刷播放只有 0.473 / 0.794**——播放量注入在现有特征上区分度最低，如实报出。

5. 业务价值评测

5.1 主结果与它的稳健版本

先说裁判是谁：浪费金额与有效曝光一律按标注结果结算，不用引擎自己的分数（eval/audit.py 中写死，产物 method.judge 字段明写用途）。否则就变成「引擎说他是水号，所以引擎省了钱」的自证循环。主口径下水号曝光按 0

计，同时给一份按 50% 计的宽松对照——两个口径同向，结论才不依赖这条假设。

单次实验（定稿种子）：三个 campaign 合计 \$245,000 预算，少浪费 \$79,719（32.5%），有效曝光 +98.64%（6,782,758 → 13,473,202），宽松口径 +64.71%。

这个数字好看，但显著性检验的结论是：它是 12 个种子里第 2 高的观测，落在均值 95% CI 之外。

	单种子	12 种子
少浪费占预算	-32.5%	-21.5% ± 13.7%（CI 12.8%~30.2%，1/12 跑输基线）
有效曝光提升（主口径）	+98.6%	+97.3% ± 54.6%
有效曝光提升（宽松口径，水号曝光按 50% 计）	+64.7%	+69.1% ± 44.3%

追加 24 种子复核进一步降到 19.1% ± 18.6%（3/24 跑输）。所以所有对外材料用的是 21.5% ± 13.7%，而不是继续挂 32.5%。

5.2 价值不在均值，在方差

同一组实验量化了另一件事：按粉丝量选人是一场高方差赌博。（口径：12 种子 × 3 campaign = 36 个观测）

	基线	KOXPilot
浪费率跨观测标准差	33.6%	4.7%（小 7.2 倍）
浪费率 > 50% 的观测	9/36（25%），最差一局 99.9%	0/36，从未超过 14.8%

决定性证据在条件胜率上——把 36 个观测按基线运气分三档：

基线浪费率	观测数	KOXPilot 胜率	平均少浪费
> 50%（基线踩坑）	9	100%（9/9）	+\$57,480
5%~50%（中等）	12	91.7%	+\$13,296
< 5%（基线运气好）	15	仅 40%	-\$2,993

KOXPilot 输的 10 个观测里，基线浪费率最高只有 5.6%——它只在「本来就没坑可踩」的局里输，且输得很小；基线一旦踩坑，9/9 全胜。

翻译成业务语言：这个产品的价值不是「帮你多赚 21.5%」，而是把「四分之一概率烧掉一半预算」这个尾部风险，换成稳定的约 4% 浪费，代价是在本来就抽到好牌的局里小输一点。对低毛利的投放代理业务，消灭尾部灾难比提高均值更值钱。

两处统计瑕疵必须自己先说，且是叠加的：

>

① 有界比例。浪费率是 [0,1] 有界比例、KOXPilot 一侧紧贴下界，尾部不正态会让 F 检验 p 值偏乐观。

>

② 伪重复。36 个观测是 12 种子 × 3 campaign，不是 36 次独立试验——同一种子下三个 campaign 共享同一个达入库。伪重复会高估自由度、进一步压小 p 值。因此合并后的 $p=2.7e-21$ 判为不可用，改报独立单位（种子）上的检验：三个 campaign 各自 $n=12$ ，方差比 p 分别为 $1.5e-07 / 1.3e-08 / 1.2e-06$ （F 值 51.6 / 81.8 / 34.9）。

>

结论因此以「标准差量级差异 + 极端频次计数（9/36 vs 0/36） + 条件胜率」为主，这些都不依赖分布假设。两条限定写在 `output/multiseed.json` → `B_variance_attribution.independence_unit`, `pooled_p_value_usable` 字段直接标 false。

5.3 价值归因：门禁占多少

原来只有两条臂（基线 vs KOXPilot），「少浪费」里混着两个原因：不把钱压在少数头部大号上（结构分散化）和识别水号并按质量排序（门禁）。为了拆开，加了第三臂 `diversified_no_gate`：同一候选池、强制 KOXPilot 的分层 / 地域 / 单人配额，但排序只看每美元名义曝光，[全程不读门禁判定与真实性折扣](#)。于是总差额 = （第三臂 - 基线） + （KOXPilot - 第三臂）。

12 种子 少浪费金额	mean ± std	为负的种子
合计	\$52.7k ± \$33.4k	1/12
其中 门禁与质量排序	\$40.0k ± \$12.6k (CV 0.31)	0/12
其中结构分散化	\$12.6k ± \$39.0k (CI 跨 0)	4/12

[稳定的那一段是门禁，不是分散投放](#)。而定稿单种子上看恰好相反（分散化 \$56.2k vs 门禁 \$23.5k）——初期判断被多种子实验推翻，该结论按实验结果保留在仓库里，未删掉重跑。

三条必须一起说的反面事实：

1. 第三臂的绝对有效曝光在 12/12 个种子上都高于 KOXPilot（它专挑 CPM 最便宜的长尾，KOXPilot 会主动放弃便宜但不对味的曝光）。[把花费归一化之后它照样赢](#)——单列为 § 5.4，不用一个分句糊过去。
2. 第三臂平均选 516 人（基线 18 / KOXPilot 181），真实采购里不可执行，它只是对照臂、不是方案。
3. 「门禁贡献」实际是「门禁过滤 + 质量排序」的合计，两者共用同一批分数，无法再拆细——所以字段就叫 `by_gating_and_quality_ranking`，不谎报成纯门禁效果。

[一处已修正的口径缺陷](#)。

界面上原先把三条臂的绝对期望值并排比。但各臂花的钱可以差几十倍（KOXPilot 只买门禁过关的人，基线会把预算花光），拿绝对数并排等于用花得多的臂衬托花得少的臂。所以主对比改成两个真正可比的字段：单位效率（`value_per_dollar`，每美元买到的有效曝光，按标注结算）与浪费金额（`waste_usd`）。绝对期望值降级为次级信息，且必须同时标明各臂花费不同。

>

举证用 § 2.4 那条自由文本需求：三臂花费 \$1,250 / \$80,000 / \$80,000（差 64 倍），每美元有效曝光 KOXPilot 53.81 / 按粉丝量 49.97 / 只分散不门禁 43.03，浪费金额 \$0 / \$4,327 / \$27,412。两处前提必须跟着说：① 这条需求花不出去钱是门禁只放过 7 人造成的，三条预置需求利用率都 ≥ 99.8%，别把 1.6% 当产品常态；② [花费量级差 64](#)

倍时「每美元效率」不能直接横比——这条需求上领先是因为只买了最干净的 4 个人；等三条预置需求上三臂都把预算花光，只分散不门禁那条臂的每美元有效曝光反超 44%~47%（§ 5.4）。这个口径比绝对数可比，但它自身也有适用前提。

5.4 对照臂在主指标上胜出：取舍与适用边界

每美元有效曝光是本项目选定的主口径（§ 5.3 即把主对比改成它的记录）。在这个口径上，diversified_no_gate 三条需求全胜，没有例外。数据来自 output/audit.json → counterfactual_value_audit.per_campaign，服务 /api/plan 的 allocation.arms 是同一批值：

需求	每千美元有效曝光 KOXPilot	只分散不门禁	差距	按粉丝量基线
BRIEF-001	56,749	83,664	+47.4%	11,964
BRIEF-002	63,359	93,454	+47.5%	64,848
BRIEF-003	50,769	73,125	+44.0%	24,230

它为什么赢。不是它作弊，也不是指标不公平——主指标由本项目自行设定，第三臂在该指标上胜出。两层原因：一是它把同一笔钱极度摊薄到微达人身上（三条需求分别选 228 / 122 / 166 人，KOXPilot 是 69 / 39 / 73），微达人 CPM 便宜得多；二是门禁会挡掉一部分「便宜且名义曝光高」的库存——资料不完整的、真实性存疑的，第三臂全程不读门禁判定，这些库存它照买。于是 KOXPilot 为更干净的库存付了更高单价。

真正的取舍。以 BRIEF-001 并排看这笔单价买到了什么：

	KOXPilot	只分散不门禁
浪费金额 / 占预算	\$2,175 / 2.72%	\$14,599 / 18.25%
名单里的水号个数	2	41
品牌安全高风险号	0	4
花在门禁会直接拒绝的人身上	\$0	\$12,390
要签、要沟通、要交付的合约数	69 份	228 份

三条需求上第三臂分别买了 4 / 2 / 4 个品牌安全高风险号（KOXPilot 是 0 / 0 / 0），分别把 \$12,390 / \$5,180 / \$14,564 花在门禁会直接拒绝的达人身上。所以结论只能这么写：门禁买的是确定性和品牌安全，不是每美元曝光效率的上限。

这件事最终让谁变好了。

1. **品牌方**：不用在投放结束后解释「为什么我们的品牌出现在争议内容旁边」（0 个高风险号 vs 4 个），也不用为 41 个水号的曝光付款——18.25% 的预算打水漂，结算时要有谁签字认下来。
2. **投放执行方**：69 份合约与 228 份合约是两种工作量。228 个微达人的报价、寄样、脚本审核、发布确认、结算，在真实代理公司里基本排不进档期。第三臂的效率是在「合约与沟通成本为零」的假设下算出来的，而这项成本不在本项目的指标体系内——因此不以此反驳第三臂，只标明这是当前指标未度量的部分。
3. **适用边界**：如果一个投放方的考核口径真的只有每美元曝光，且品牌安全与水号曝光都不重要，那这套东西不该被采用，应该直接用第三臂那种最大化分散度的策略。它压的是不确定性与品牌风险，只在「翻车有代价」的场景里值钱。

BRIEF-002 上还有一处更不利的细节，不省略。该需求上 KOXPilot 浪费金额 \$5,494（12.21%）高于第三臂的 \$3,695（8.21%），有效曝光率也更低（87.11% vs 91.55%）。这说明门禁在那条定向下没有起到应有作用——3 个水号进了最终名单，是真实待改进项，不是口径问题。同一条需求上按粉丝量基线的每千美元有效曝光 64,848 也高于 KOXPilot 的 63,359，浪费金额只有 \$376（0.84%）——所以「至少稳定赢按粉丝量基线」这一表述在 BRIEF-002 上同样不成立，故不予采用。这条需求上唯一站得住的事实是买入库存的干净程度：基线把 \$44,624 花在门禁判「需人工复核」的人身上，KOXPilot 是 \$0。

一个还没做的下一步。两条臂的策略并不互斥：先用门禁过滤掉复核 / 拒绝档，再在干净池子里最大化分散度（而不是像现在这样按质量加权价值排序），理论上可能同时拿到干净和效率。该方案尚未实现，也没有对应实验数据——列在这里是标明方向，不计入成果。

6. 大模型的使用判据与成本

6.1 该不该用模型：用实验回答

「用了大模型」本身不是能力证明。所以做了一组控制变量实验：同一批 600 样本、同一标注、同一 batch 切分，唯一变量是 system prompt。任务选最需要语义理解的那一层：标签错配识别（声称做 3C，实际在做什么）。

Arm	Precision	Recall	F1	token
零成本规则对照组	0.4190	0.8571	0.5625	0
Prompt v1（角色 + 任务描述）	0.2367	0.8453	0.3627	434,018
Prompt v2（加约束与 few-shot）	0.3554	0.7857	0.4889	467,327
Prompt v3（业务判据化）	0.6136	0.7976	0.6905	492,460
最优组合 v3 × GPT-5.5	0.6875	0.7857	0.7333	—

结论是反直觉的：v1 和 v2 都输给零成本规则。花了 90 万 token，做得比一个 Jaccard 规则还差。v3 反超的原因不是提示词写长了，而是把它从「你是一个专业的内容审核专家，请判断……」改成业务判据：写清什么叫错配（主品类漂移 vs 兼做副品类）、给出可判定的边界、要求先输出证据再输出结论。

对产品的意义：LLM 在这条链路上的价值不取决于模型多强，取决于能不能把业务判据写清楚。判据写不清，模型不如规则；判据写清了，模型比规则高 13pp。

6.2 成本账与分工

方案	LLM 调用数	代价
全 LLM（每层每人都问模型）	888	贵、慢、不稳定
KOXPilot 混合（G0/G1 纯规则，仅需求解析与语义适配用 LLM）	225	调用量降 3.95×
全规则（不用 LLM）	0	标签错配 F1 从 0.6824 掉到 0.5625（-12pp）

真实 token 账（取自各 API 返回的 usage，非估算）：[172 次调用](#)、[110 万 token](#)（prompt 53.7 万 / completion 56.7 万，其中 reasoning 40.3 万）。其中有 1 个 batch 调用失败，按「未覆盖」处理而不是填默认值——填默认值等于伪造模型输出。

分工口径必须写清：标签错配判定与 A4 适配分是[构建期真调模型](#)、[结果固化进产物](#)，服务不重算这两处；服务运行时唯一可能调模型的是 A1（§ 3.4）。A4 的 LLM 版为什么没进正式链路：780 条真实调用得到的适配分，落到三个 campaign 的去重候选池（222 人）里只覆盖 33 人（[14.9%](#)），与既有筛选双算率 97.1%，覆盖子集上分数只降不升——按判据不予升格，且判据是可自动翻转的（覆盖率 ≥ 90% 等三项全过会自动改口径）。

7. 工程完备性与可复现性

测试	768 条 pytest 全绿 ，含防自证元测试（AST 扫描确认生产代码读不到标注）、HTTP 接口契约测试、文档数字防漂移测试
确定性	固定种子，make eval 两次运行 metrics.json 逐字节相同 （产物刻意不含时间戳，任何指标变化都可归因到代码或数据）
双实现一致性	见 § 3.2（运行期 1,431 条 0 差异、构建期 5,000 条 0 差异）
服务与离线问题	契约 v1.1 + 只读 scope 块（§ 3.3）
假设可审计	预算模型里 decay=0.7 是 建模假设不是观测值 ，补了三档敏感性扫描

7.1 假设敏感时的分层交付

decay 扫描的结论坚持两半一起说：[价值结论不依赖这个假设](#)（合计少浪费相对离差仅 1.4%，两段归因符号逐档不翻转），[但具体选谁会变](#)（选中名单金额加权重叠最低 0.651，低于自设的 0.8 判据）。产物里有专门字段与测试堵住「已验证 0.7 是对的」这种过度解读——这条扫描回答不了哪一档更接近真实世界，那需要真实投放的重复触达数据。

判据不达标之后，光承认「名单不唯一」是不够的——投手拿到的仍是一份不知道哪里靠不住的清单，不确定性被承认了，成本却全转嫁给使用者。所以接了一条降级路径：按「在几档 decay 上都被选中」把名单切成两层。

交付层	人数	参照档金额	占比	动作
核心层（三档全选中）	162	\$220,855	90.2%	可直接下单，入选不依赖 decay 取值
假设敏感层（仅部分档位选中）	19	\$23,962	9.8%	标记「重复触达折扣假设敏感」，转人工确认或改按单条采购，不自动执行

结论因此从「名单不唯一」变成一句可执行的话：[这个假设的不确定性只影响 9.8% 的预算，且已定位到具体 19 个人](#)。产物见 `output/metrics.json` → `budget_decay_sensitivity.delivery_policy`，两层必须无损划分正式名单、金额之和必须等于实付金额，由 `tests/test_decay_delivery_tiers.py` 钉住。

8. 已知弱项与边界

弱项	数字	判断
数据是合成的	—	最大边界。所有绝对值只在数据生成假设内成立。唯一能推翻或确认结论的实验是接一批真实达人 + 人工标注重跑
头部大号识别差	follower_bucket=mega F1 0.467	系统性短板 （11/12 种子进榜， $p=1.5e-07$ ），不是噪声
单次弱项清单不可信	country=SA、youtube macro	是 采样噪声 （ $p=0.132 / 0.803$ ）。典型 winner's curse：单次实验里最差的几格，多数只是运气差
阈值敏感性不稳健	±20% 最大 F1 偏移 0.0582，stable=False	如实上报，没有调宽判据让它变 True
刷播放最难抓	严召回 0.473	播放注入在现有特征上区分度不足
A4 的 LLM 版未进正式链路	覆盖率 14.9%	口径明确为「规则版在生产链路、LLM 版构建期跑通并固化」，不让文档继续说 LLM
A1 在演示实例走规则解析	llm_runtime.available=false	演示环境未配置模型凭据，服务把降级原因逐字写进响应。配置凭据重启即走 LLM 路径，是部署配置问题，不是功能缺失
BRIEF-002 上门禁没起到应有作用	3 个水号进名单，浪费 12.21%	真实待改进项：该需求上浪费率高于对照臂（8.21%）与基线（0.84%），有效曝光率 87.11% 三臂最低。是 G1 在那条定向下没抓住，不是口径问题（§ 5.4）
服务是演示实例	—	单实例、无副本、无灰度、只读不写，冷启动首个请求需等懒加载
名单不唯一	金额加权重叠最低 0.651	换 decay 假设就换一批人。已接分层交付，不确定性收敛到 9.8% 预算、19 人（§ 7.1）

弱项	数字	判断
预算分配非最优	—	「边际性价比贪心 + 结构约束修正」，叠加 5 条约束后不保证最优，没做 MILP 对照，所以说不出离最优差多少

9. 迭代记录：被实验推翻的判断与修复

仓库 README 完整列了 14 条自查与修复，这里挑三类最能说明问题的。

9.1 第一类：指标漂亮但来自自证

问题	表现	修完的代价
标签错配的标注实际等价于「两个集合是否相交」	规则基线 $P=R=F1=1.000$	引入隐藏真实品类后，规则基线 F1 掉到 0.562
观测模型整块替换，正例 Jaccard 只有 0 和 1	5 个 arm 召回精确相同 (0.857)，尽管彼此预测差异 146/600	加逐品类观测噪声后，召回重新成为真实 tradeoff
消融表里有一句能被当场证伪的话	contributes 只看 abs(delta) 不看符号，于是关掉 G1.6 后 F1 反而涨 0.0149 的规则被算成「有贡献」，产物还写着「所有 G1 规则都有边际贡献」	改三态判定 (positive / negative / negligible)， 保留 G1.6 并如实标为负贡献
一个会爆炸的比率	有效曝光提升的分母是基线有效曝光，基线烧光时分母趋 0，实测出现 $+3585\times$ ，把跨种子均值抬到 $+29,977\%$ (中位数仅 $+124.5\%$)	改用有界口径 (有效曝光率百分点差)，保留旧字段并标注不稳定原因
「用 LLM 做语义适配」这句话在正式链路里站不住	缓存里有 780 条真实 LLM 适配分，而 make eval 的每个数字都来自规则版	补量化审计 (覆盖率 14.9%、双算率 97.1%)，结论「继续用规则版」 由判据算出 ，并真跑了注入反事实 (判定只翻 1 条、少浪费 \$735，不作为招牌数字)

这几条的共同点：[它们都不会报错，只会给出一个漂亮得不真实的结论](#)。由此固化下来的检查动作是——指标好得不合理时，优先排查自证路径；多个方法结果完全相同时，先检查可分性是否已经退化。

9.2 第二类：结论正确，但没有产生动作

其一：敏感性扫描判出 `selection_stable = false`，原处理是在文档里补一句「名单会随假设变动」。这句话没错，但对使用者没有用——不确定性被承认了，成本全转嫁给了投手。修法是 § 7.1 那张分层交付表：把不确定性从「整份名单打个折扣」收敛成「具体 19 个人、9.8% 的预算」。同一份数据，一个是免责声明，一个是分流规则。

其二：`decay_scan.py` 里把记录分成 blockers (会推翻结论的硬伤) 与 caveats (不推翻结论但必须一起说的限定)，源码注释还专门解释了混成一个列表的危害——[但返回值里根本没有 caveats 这个键，算完就被丢掉了](#)。本次实测 blockers 为空而 caveats 有 1 条 (金额加权重叠 0.651 不达标)，读者只会看到一片绿。修法：把 caveats 接回产物并与 blockers

分列两个键，由测试断言「selection_stable=false 时 caveats 不得为空、两类记录不许混装」。

其三：README 里写过一句「没有一个数字是手写进文档的」。这句经不起追问——数字当然是手打进 markdown 的，只是来源于产物；真正的风险是产物重跑后数值变了而文档里的字不会自己改。修法不是把话说软，而是补 tests/test_doc_numbers.py：把文档里的招牌数字逐条钉到产物字段上，连千分位、百分号、有效位都按人眼看到的字符串断言，并包含一条「文档声称的测试总数必须等于实际收集到的数量」。不改口气，改机制。

这三条比第一类更值得说。第一类是技术功底问题，这三条是产品意识问题：一个结论如果不能推导出下一步动作，那它在产品里就还没做完。

9.3 第三类：展示参数改变了计算语义

这条性质又不一样——代码不报错，响应结构完整，数字看起来还算合理，但服务和离线在算两道不同的题。

- **症状**：同一条 BRIEF-001，离线命令行选 69 人花 \$79,855（利用率 99.82%），线上服务只花 \$5,693（7.12%）——预算利用率差 14 倍。
- **根因**：服务的 _recall 把 options.top_n（一个展示用参数，默认 200）当成了召回上限。BRIEF-001 命中定向 746 人，被截到 200 之后，这个残缺的池子直接进了门禁和分配器；而分配器的结构配额（长尾 $\geq 25\%$ / 单国 $\leq 60\%$ / 单人 $\leq 20\%$ ）是在候选池上求解的，小池里「买得起又能满足配额」的库存跟着消失，于是无可行解，钱花不出去。
- **修法**：截断只作用于展示——top_n / explain_limit 只裁返回的候选明细条数，召回一条不裁。服务升 1.1.0、契约升 v1.1，并新增只读 scope 块把「参与计算的候选没被裁」写进响应体（§ 3.3）。src/koxpilot/一行未改：错的一直只是服务层那一次切片，768 条测试也因此全程是绿的。
- **教训**：这类 bug 只有把两条路径的中间量逐个对齐才能发现，看结果对不对是发现不了的。服务返回的清单、金额、约束检查全都自治，all_enforced_satisfied 甚至是 true；只有把服务与命令行的候选池条数摆在一起，才看出一边 746、一边 200。所以修法不止改那行切片，而是把中间量（recall_total / computed_on）提成响应里的一等字段——下次再出这种事，它得自己露出来。

10. 交付说明

10.1 交付形态

正式交付只有一个 [GitHub 仓库](https://github.com/2990709874-ui/koxpilot)：https://github.com/2990709874-ui/koxpilot

下面这些都在仓库里，不需要再去别处找：

内容	位置
本文档 PDF	仓库根目录 AI 应用能力文档_KOXPilot_纪辉.pdf

内容	位置
在线 Demo	https://2990709874-ui.github.io/koxpilot/ （已由 .github/workflows/pages.yml 发布，浏览器直接打开即可交互；也可本地一条命令起）
HTTP 服务	api/，契约在 api/CONTRACT.md，公网部署包在 deploy/hf-space/
业务内核与评测	src/koxpilot/（6 个 Agent + 评测 + 审计）
全部指标产物	output/（已入库，clone 完不跑任何命令即可核对本文档每个数字）
合成数据与需求样本	data/（已入库，固定种子，make data 可逐字节重建）
六篇技术分册	docs/01 架构 docs/02 四层门禁 docs/03 评测方法 docs/04 Prompt 与成本 docs/05 边界与已知问题 docs/06 稳健性

10.2 三条复现路径

① 数据 → 门禁 → 预算 → 评测（固定种子，无网络依赖，约 1 分钟）

```
pip install -e .
make all
make test # 768 条测试
make multiseed # 12 种子稳健性实验（纯确定性，约 30s）
```

② 前端 Demo（不依赖任何外部服务）

```
cd web && pnpm install && pnpm run prepare-data && pnpm run dev # → http://localhost:5173
```

③ HTTP 服务（无鉴权，可直接 curl）

```
pip install -r api/requirements.txt
cd api && python3 main.py # → http://127.0.0.1:8000
curl -s http://127.0.0.1:8000/api/health
curl -s http://127.0.0.1:8000/api/kox/KOX-000002/explain
curl -s -X POST http://127.0.0.1:8000/api/plan \
  -H 'Content-Type: application/json' \
  -d '{"brief_text": "巴西和墨西哥的 Instagram 彩妆, 18-24 女性, 预算 8 万美金, 避开 Focallure"}'
```

可选：需要模型凭据的构建期实验

```
make llm # 真实 LLM runner（A1 解析 + 标签错配 + 适配打分）
make promptbench # Prompt v1/v2/v3 × 双模型横评
```

两点提示：api/run.sh 不是本地启动脚本（它由部署平台生成，里面会 cp -r /opt/bytfaas/*，只在部署环境成立）；把 api/_bundle/ 整个移走也能跑通，路径解析是「api/_bundle 优先、回落仓库根」，clone 下来没有部署产物的情形同样成立，实测 BRIEF-001 仍返回 69 人 / \$79,855.29。

10.3 要让前端接上公网服务 / 走 LLM 路径

1. 按 [deploy/hf-space/README.md](#) 把服务部署到公网（Hugging Face Spaces 一键包，已本地验证）。
2. 改 web/public/api-config.json 的 apiBase 指向该地址，前端无需改代码重构建。

3. 需要 A1 走 LLM：给服务环境配置 ARK_API_KEY 或 Azure OpenAI 相关变量（模板见 .env.example），重启后 /api/health 的 llm_runtime.available 会变成 true，/api/plan 的 brief.parse_path 变成 llm。凭据不入库。

附录：产物索引

产物	内容
output/metrics.json	六张指标表 + 反事实审计 + 诚实性清单
output/audit.json	按标注口径的三臂价值审计
output/multiseed.json	12 种子稳健性与方差归因
output/prompt_bench.json	Prompt v1/v2/v3 × 双模型横评
output/llm_bench.json	真实 token 账（172 次调用 / 110 万 token）
web/public/data/consistency.json	Python 与 TypeScript 双实现全库比对（5,000 条 0 差异）